

C++基础：基本控制结构 与导出数据类型

目录

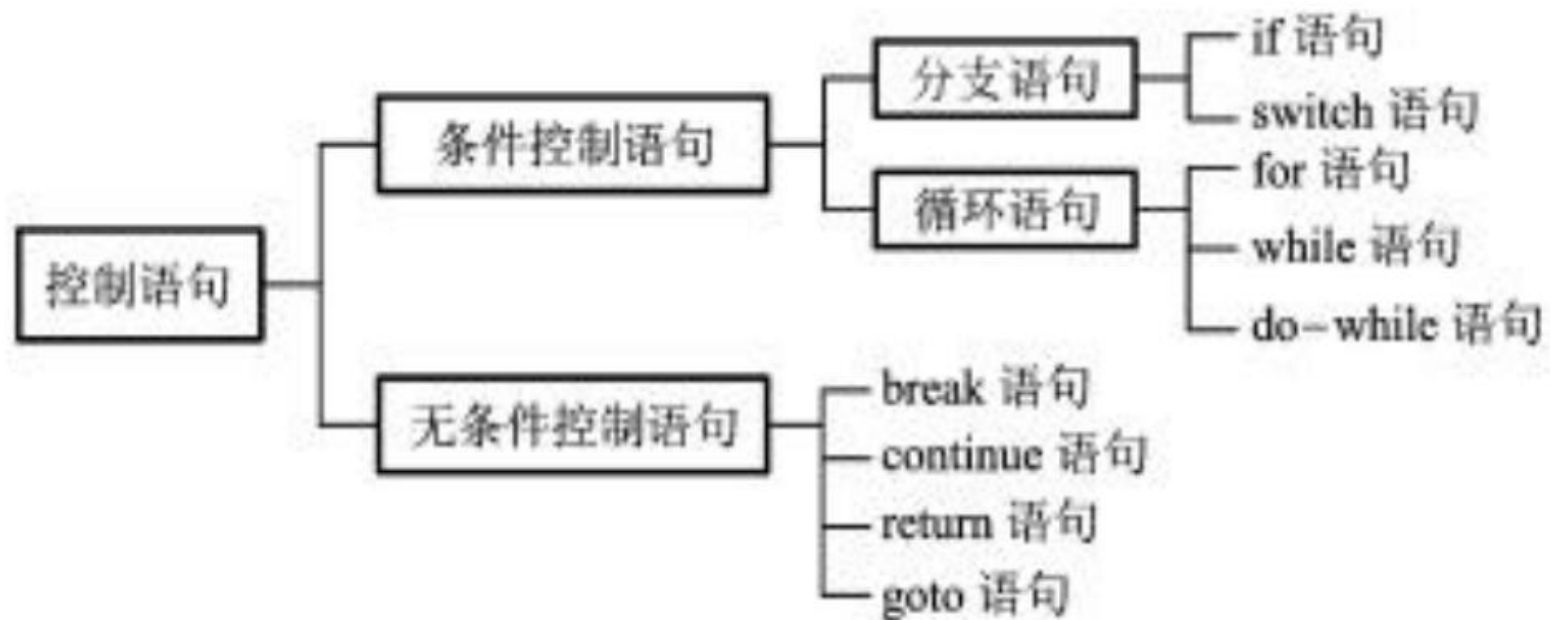
- 基本控制语句
 - 分支语句
 - 循环语句
 - 转向语句
- 导出数据类型
 - 数组
 - 结构类型

简单程序结构与控制语句

- 程序设计或编程就是对一系列要计算机完成的命令的先后次序进行安排，如果不加任何控制结构，就是单纯按书写顺序执行的程序，是**简单结构程序，其执行规则为：执行顺序=书写顺序**
- **简单程序实际上几乎没有实用价值。**首先无“智能”可言，它不会根据情况进行判断，决定不同的处理，所有的操作顺序都需要程序员在编程时安排好。其次，简单程序写多长的程序就执行多长，对于目前每秒钟能完成上亿次浮点运算的计算机，程序的长度难以计算，编程将变成人们无法完成的任务。（想象一下求1到1万亿的和）
- 因此，我们需要控制语句来改变程序的这种简单结构。例如，分支语句可以根据程序执行到某一步的具体情况决定下一步转向不同的程序段；循环语句则可以令计算机反复执行某一段程序多次，有了循环语句，才使得一些不很长的程序却可能在高速计算机上运行很长时间。

控制语句

- 控制语句用来控制程序中各语句执行的次序，更确切地说，控制语句可用来**改变或打破 程序中按语句的先后次序顺序执行的规律**。
- C++语言中的控制语句分条件控制语句和无条件控制语句，如图所示：



- 包含控制语句的**计算机程序的执行过程一般称为程序的动态结构，而程序的书写顺序称为静态结构**。程序的动态结构与静态结构的差别，给程序设计、调试和维护造成了困难，特别是当程序的规模较大时，软件的开发与维护是一项非常复杂的工作

复合语句

- 复合语句，又称为块语句（block）或程序块，它是若干语句的组合，其格式为：

```
{<语句 S1>  
  <语句 S2>  
  ...  
  <语句 Sn>  
}
```

- 复合语句，与说明语句、表达式语句、控制语句不同，构成复合语句的 S_i ：可能是说明语句、表达式语句、控制语句、也可能是一个复合语句（嵌套）。
- 复合语句虽然包含多个语句，但在逻辑或概念上，它作为一个语句出现。复合语句可以出现在程序中的任何地方。

目录

- 基本控制语句
 - 分支语句
 - 循环语句
 - 转向语句
- 导出数据类型
 - 数组
 - 结构类型

分支语句-if语句

- 分支语句亦称选择语句属于**有条件控制语句**，它是计算机按条件进行判断和“思维”的基础。C++提供两类分支语句，if语句和switch语句。
- **if**语句也称条件语句或如果语句，它有两种形式：

if (<表达式 **E**>) <语句 **S**>

if (<表达式 **E**>) <语句 **S₁**> **else** <语句 **S₂**>

例如：

if (a<= b) {c=a;a=b;b=c;}

if (a<c) cout<<a; else cout<<c;

分支语句-if语句

- **表达式 E:** 在 if 后面, 用 (,) 括起来, 该表达式应为 int 型或 char 型的表达式, 因此, int 型、long 型、char 型以及 enum 型表达式都是合法的, 其值等于 0 为假, 非 0 为真。
- **语句 S:** 可以是任何类型的语句, 也可以是块语句。语句 S1、语句 S2 同 S。
- **关键字 else:** 出现在第 2 类 if 语句中, 它表示当表达式 E 取假值 (0) 时跳过语句 S1, 执行 S2。

if (<表达式 E>) <语句 S>

if (<表达式 E>) <语句 S₁> else <语句 S₂>

例如:

if (a <= b) {c=a;a=b;b=c;}

if (a < c) cout << a; else cout << c;

分支语句-if语句

- 严格地说，表达式 **E** 应为一个布尔表达式，其值应为 **false**和 **true**。C++语言对 E 的要求较松，比较方便灵活。
- if 语句中的语句 S、S1、S2 也可以是 if 语句，即 if 语句的嵌套。在 if 语句嵌套时，C++语言规定，**为了避免二义性，每个 else 只与其前面最近的未配对的 if 配对。**

if (<表达式 **E**>) <语句 **S**>

if (<表达式 **E**>) <语句 **S**₁> **else** <语句 **S**₂>

例如：

```
if (a <= b) {c=a;a=b;b=c;}
```

```
if (a < c)    cout << a; else    cout << c;
```

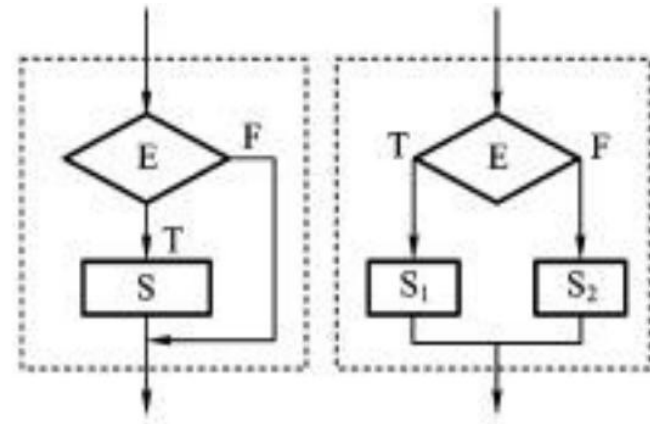


图 4.2 两类 if 语句的流程图

分支语句-if语句

Code

```
5 void if_test(){
6     int a, b, c, max;
7     cout << "first integer:";
8     cin >> a;
9     cout << "second integer:";
10    cin >> b;
11    cout << "third integer:";
12    cin >> c;
13    if (a<b) // 以下 6 行程序用来求最大元
14    |       if (c<b) max = b;
15    |       else max=c;
16    else
17    |       if (c<a) max = a;
18    |       else max = c;
19    cout <<"maximum element: " << max << endl;
20 }
```

Run

first integer:23	first integer:456
second integer:65	second integer:213
third integer:123	third integer:78
maximum element: 123	maximum element: 456

分支语句-switch语句

- switch 语句又称开关语句或分情况语句，它是一种多分支语句。其基本格式为：

```
switch (<表达式 E>) {  
    case <情况常量 C1>: <语句序列 S1>  
    case <情况常量 C2>: <语句序列 S2>  
    ...  
    case <情况常量 Cn>: <语句序列 Sn>  
    default: < 语句序列 Sn+1>  
}
```

- 表达式 E: int 型、char 型及其派生类型和枚举类型的表达式。
- 关键字 case: 用来引出一个分支。
- 情况常量 C_i: 为表达式 E 的一个可能值。当计算表达式 E 的值为 C_i 时，控制跳过前面的 S₁, ..., S_{i-1} 语句序列，直接执行语句序列 S_i, ..., S_n 语句序列。

分支语句-switch语句

- **default:** 关键字 default 表示当表达式 E 的值不等于已列出的情况常量 $C_1, C_2, \dots, C_n$ 时, 控制直接转向语句序列 S_{i+1} (跳过 $S_1, S_2, \dots, S_n$)。 **default 分支可以缺省, 它相当于 S_{n+1} 为一空语句。**
- 语句序列 $S_1, S_2, \dots, S_{n+1}$ 是 switch 语句的各个分支, S_i 可以是任意多个语句的序列。

```
switch (<表达式 E>) {  
    case <情况常量  $C_1$ >: <语句序列  $S_1$ >  
    case <情况常量  $C_2$ >: <语句序列  $S_2$ >  
    ...  
    case <情况常量  $C_n$ >: <语句序列  $S_n$ >  
    default: < 语句序列  $S_{n+1}$ >  
}
```

分支语句-switch语句

- 表达式 E 和情况常量 $C_1, C_2, \dots, C_n$ 应为同一类型, 且 $C_1, C_2, \dots, C_n$ 互不相同, 当需要对表达式 E 的若干不同值对应于同一语句序列时, 可把它们列在一起:

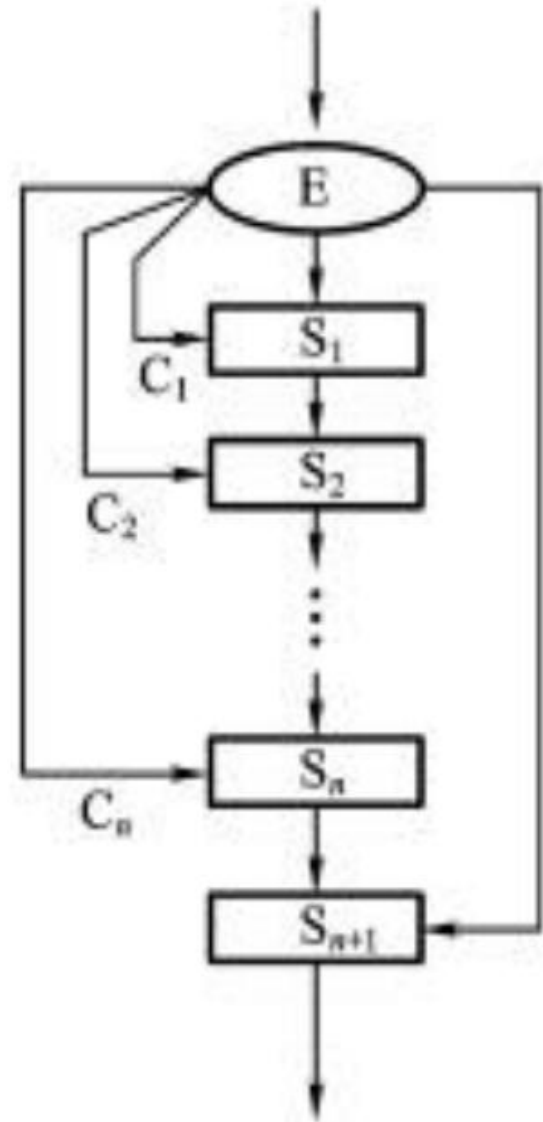
case C_1 :

case C_2 :

case C_3 : S_3

这样就可实现了当表达式取 C_1 、 C_2 或 C_3 时, 都转向语句序列 S_3 。

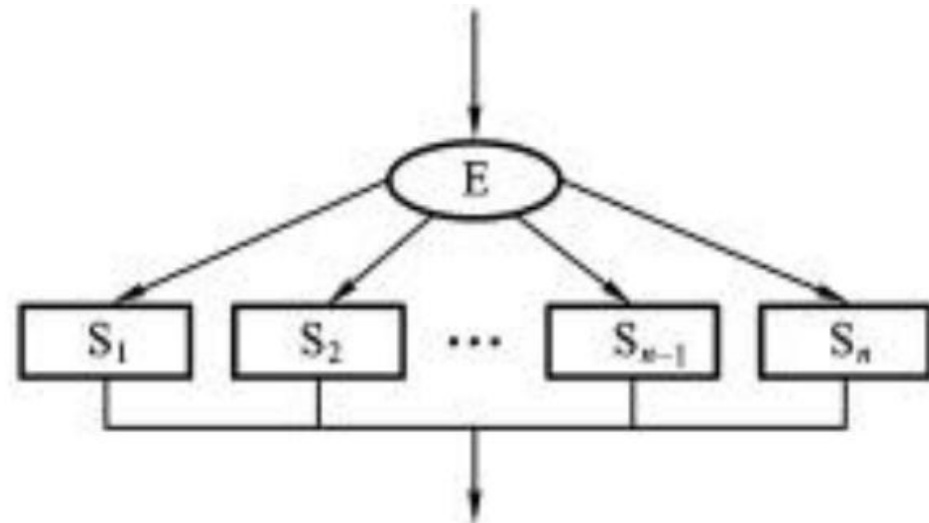
- 注意右图给出的流程图, 当表达式 E 取值 C_i 时, 控制转到语句序列 S_i (跳过 $S_1, S_2, \dots, S_{i-1}$), 在执行完 S_i 之后, 不是跳出整个开关语句, 而是继续执行语句序列 $S_{i+1}, \dots, S_n, S_{n+1}$, 然后再转出开关语句继续执行后面的程序。



分支语句-switch语句

- 如果实际的问题要求程序在执行各情况分支之后，直接跳出开关语句（见图 4.4），则开关语句的使用需略作改动，即**在每个情况分支的语句序列之后增加一个 `break` 语句**。

```
switch (E) {  
    case C1: S1  
        break;  
    case C2: S2  
        break;  
    ...  
    case Cn: Sn  
        break;  
    default: Sn+1  
}
```



分支语句-switch语句

Code

```
26 void switch_test()
27 {
28     float t, tc, tf; char corf;
29     const float fac = 1.8, inc = 32.0;
30     cout << "Enter temperature:";
31     cin >> t;    // 输入温度值
32     cin >> corf; // 指出是摄氏(c)或华氏(f)
33     switch(corf){
34     case 'c':
35     case 'C': tc = t; tf = t * fac + inc; break;
36     case 'f':
37     case 'F': tf = t; tc = (t - inc) / fac; break;
38     default:
39         tc = tf = 0.0;
40     }
41     cout << "The temperature is:";
42     cout << tc << "C=" << tf << "F\n";
43 }
```

Run

```
Enter temperature:43c
The temperature is:43C=109.4F
```

目录

- 基本控制语句
 - 分支语句
 - 循环语句
 - 转向语句
- 导出数据类型
 - 数组
 - 结构类型

循环语句

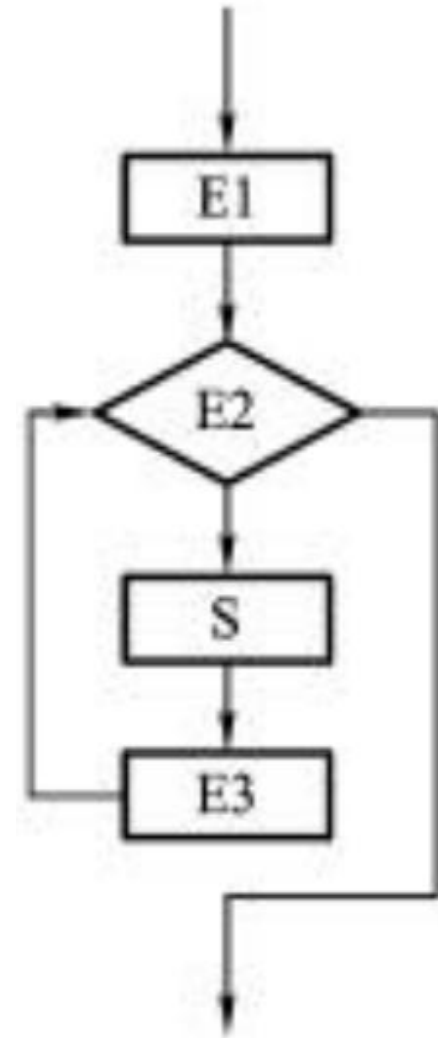
- 在实际应用中，人们常需要计算机重复多次完成相同或相似的动作，这时，使用循环语句就非常有效。正是由于循环语句的使用，使得人们可以写出很短的程序令计算机做大量的工作。
- 循环语句亦称重复语句，它可以按一定规则控制一段程序（循环体）重复执行若干次。
- C++语言提供 3 种循环语句：**for**，**while**，**do-while**

循环语句-for语句

- for 语句是 C++ 程序中最常用的、功能最强的循环语句，其格式为：
for（<表达式 **E1**>; <表达式 **E2**>; <表达式 **E3**>）<语句 **S**>
- **表达式 E1**：一般用于为循环控制变量赋初值，故又称初值表达式，但从语法规则看它并不一定是一个赋值表达式。
- **表达式 E2**：用于判定循环是否继续，称为条件表达式，E2 必须具有整型值的 **int** 型、**char** 型及其派生类型和枚举类型的表达式。
- **表达式 E3**：一般用于循环变量的增（减）值，故称为增量表达式，但从语法规则看它可以是任何表达式语句。

循环语句-for语句

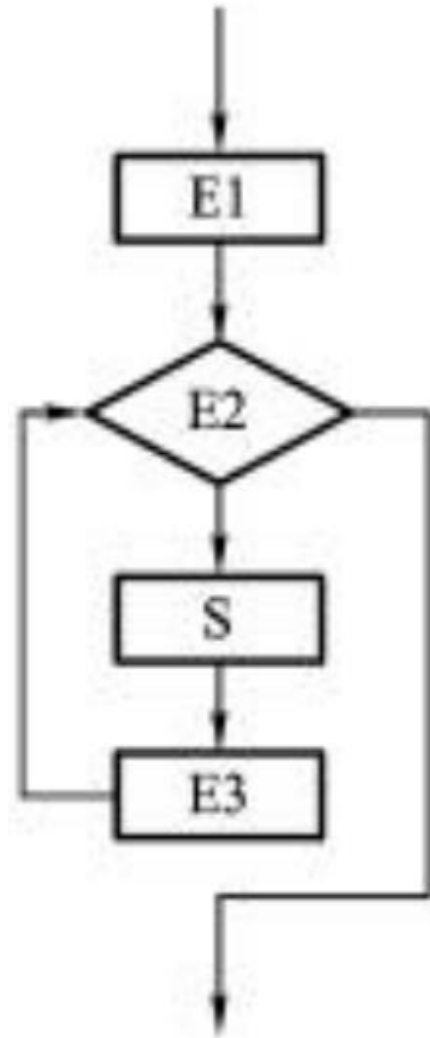
- for 语句的流程图如右图所示。
- **语句 S**：它可以是单个语句，也可以是（在多数情形下）**块语句**，它是要被循环重复执行的程序段，故又称为**循环体**。
- 表达式 E1、E2、E3 都可以为空。其中 **E2 为空语句**，意味 **E2 永远取值为真**，循环体将可能无限循环下去，除非 S 中含有在一定条件下执行跳转的语句。
- 表达式 **E1** 只执行一次，不仅可初始化循环变量，也可使用逗号表达式，初始化其他成分。
- 表达式 **E2** 和 **E3** 在每次执行 S 的前后要分别执行一次，如果采用逗号表达式的形式，则 E2、E3 除了完成判定循环条件和循环变量增量的功能之外，还可进行其他数据处理。



循环语句-for语句

- S 可以是空语句，可用于某种延时程序。

for (t=0;t<some-value;t++);

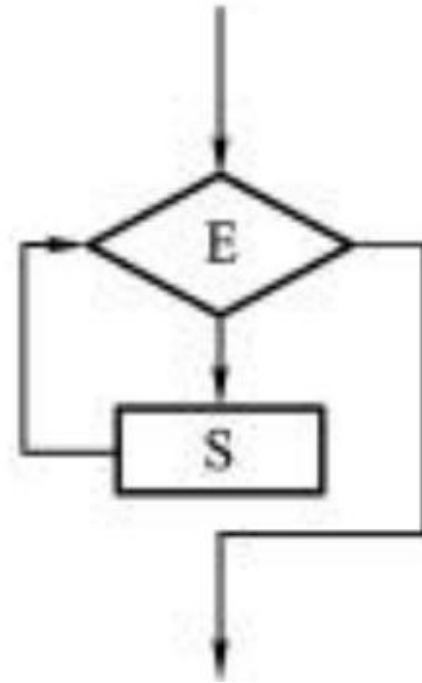


循环语句-while语句

- while 语句是一种形式简单的循环语句，它实际上是for 语句在表达式 E1和 E3 为空的特殊情形。其流程图如右所示。其格式为：

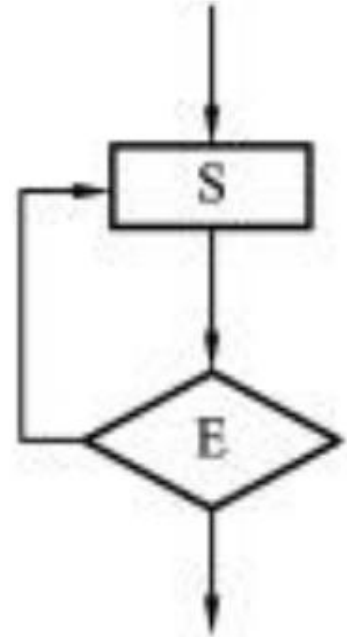
```
while (<表达式>) 语句 S
    while (inp!='Q') {
        cin>>inp;
        cout<<inp*inp;
    }
```

- 表达式 E：用于判定循环是否继续，首先计算 E 的值，为非零整数即为真，执行循环体，循环体执行后再计算 E，直到 E 取 0 值（假）时跳出循环。语句 S：为循环体。
- 不难看出：“while (E) S” 等价于 “for (;E;) S”



循环语句-do-while语句

- do-while 语句类似于 while 语句，只是把循环的判定移至循环体之后，如右图所示，其格式为：
do <语句 **S**> **while** (<表达式 **E**>) ;
- 语句 **S**：为循环体。表达式 **E**：判定循环是否继续，其值为真（非 0 整数）时返回执行循环体 **S**，否则跳出。
- do-while 语句与 while 语句的主要区别是，它的循环体至少被执行一次；而 while 语句首先判断条件，可能一次也不执行就跳出。（do-while 也不是 for 循环的一种特例）



实例：求素数

```
53 void find_prime(int m)
54 {
55     int i, j, isprime, count=0;
56     for (i = 2; i <= m; i++)
57     { isprime = 1;
58       for (j = i - 1; j > 1; j--) // 判断 i是否为素数
59       { if (i % j == 0)
60         isprime = 0; // 用 j(i-1~2)试除 i
61       }
62       if (isprime){
63         cout << i << ", ";
64         count++;
65         if (count % 20 == 0)
66             cout << endl;    } }
67     }
```

2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,
73,79,83,89,97,101,103,107,109,113,127,131,137,139,149,151,157,163,167,173,
179,181,191,193,197,199,211,223,227,229,233,239,241,251,257,263,269,271,277,281,
283,293,307,311,313,317,331,337,347,349,353,359,367,373,379,383,389,397,401,409,
419,421,431,433,439,443,449,457,461,463,467,479,487,491,499,503,509,521,523,541,
547,557,563,569,571,577,587,593,599,601,607,613,617,619,631,641,643,647,653,659,
661,673,677,683,691,701,709,719,727,733,739,743,751,757,761,769,773,787,797,809,
811,821,823,827,829,839,853,857,859,863,877,881,883,887,907,911,919,929,937,941,
947,953,967,971,977,983,991,997,

目录

- 基本控制语句
 - 分支语句
 - 循环语句
 - 转向语句
- 导出数据类型
 - 数组
 - 结构类型

转向语句

- 转向语句指可以直接控制程序跳转的语句。由于C++程序，默认是按顺序执行和优化的，转向语句打破了顺序执行的规则，给出了可以在程序任意位置进行跳转的能力，可以提升程序灵活性，但是随意使用可能会导致意想不到的错误和性能瓶颈。
- C++语言提供了 4 种无条件转向语句：**break**，**continue**，**return**，**goto**。

转向语句

- **break** 语句又称跳出语句。它用于循环语句和 switch 语句，其功能是把程序的运行流程 跳转到所在的循环语句或 switch 语句的出口。其格式为：

break;

仅由一个关键字 **break** 组成，使用方便，含义也很明确。

- **continue** 语句又称继续语句，是仅用于循环语句中的一种控制语句。其功能是把程序运行流程跳转到该循环体（不是该循环语句）的出口点，结束循环体的一次重复，其格式为：

continue;

continue 语句和 **break** 语句类似，虽然都是无条件跳转语句，但其跳转的范围明确，不易产生问题，是推荐使用的无条件转移控制语句

转向语句

- **return** 语句又称返回语句。只用于函数定义，其功能为：把程序运行的流程跳转到该函数的调用点或函数调用的出口点。计算返回表达式 *E* 并把其值作为该函数的返回值

- **return** 语句格式为：

return; 或 *return*<表达式 *E*>;

- 表达式 *E*：当函数的返回类型为 `void` 型时，返回语句应取第 1 种方式，否则，取第 2 种方式。表达式 *E* 的类型应与函数的（返回）类型相一致。对于非 `void` 类型的函数来说，其函数体中的 **return** 语句是必不可少的。

转向语句- goto语句

- goto 语句又称转向语句，其功能是令程序跳转到程序指定的某标号语句处。由于标号语句的设置较为灵活，因此，goto 语句的跳转控制比上面 3 种跳转语句有较大的随意性。其格式为：

goto <标号>;

- 标号是一个标识符，其定义出现在标号语句： <标号>: <语句>

L1: <语句 S1>	L2: <语句 S2>
...	...
goto L2;	goto L1;
...	

- 语句 goto L2;把控制流程跳转到 L2: <语句 S2>, 而语句 goto L1;又跳转到 L1: <语句 S1>。标号语句可以出现在转向它的 goto 语句之后，也可以出现在其之前。goto 语句和 break、continue、return 语句都是无条件转移语句，无需任何判断，程序执行到此时，必须跳转到一个确定的位置。

转向语句- goto语句

- **goto** 语句转向的目标位置由程序员任意确定，而 **break**、**continue** 和 **return** 语句的转向目标位置由其本身的位置所决定。然而，**goto**语句的转向目标——标号语句的位置虽然可由程序员确定，但不是完全任意的，它应满足下面的限制：
 - 一个**函数定义（函数体）内的 goto 语句不可转到函数之外**。换句话说，函数的出口点只能是 **return** 语句或函数体结束。
 - 一个**块语句（包括函数体和循环体）外的 goto 语句不可转到该程序块之内**，因为这往往会产生计算机难于处理的局面。
- 基于**goto**语句可以实现任意的程序控制代码，包括但不限于分支语句、循环语句、转向语句等等，基于**goto**几乎可以实现任意你想要实现的流程控制。

目录

- 基本控制语句
 - 分支语句
 - 循环语句
 - 转向语句
- 导出数据类型
 - 数组
 - 结构类型

导出数据类型

- 导出数据类型并不是新的数据类型，而是定义在其他已定义类型基础上的类型，而且其运算也是确定的。C++语言中，导出类型主要有 3 种：
 - **数组类型**：若干同一种类型数据（或对象）的组合。
 - **指针类型**：某一种类型数据（或对象）的首地址。
 - **引用类型**：某一种类型数据（或对象）的引用。
 - **结构（及联合）类型**：由若干不同类型数据（或对象）组合而成，
- 哈哈

数组类型

- 在计算机所处理的数据中，最需要由计算机高速处理的数据是成批出现的**同一类型的数据**，C++语言中把这类数据称为数组。
- 数组（**Array**）不是一种独立的数据类型，称之为导出类型。任何一种类型的一批数据，都可以组成数组，例如 **int** 型、**float** 型、**char** 型数据可以组成数组；这些基本类型的派生类型（如 **long** 型，**double** 型）的数据，枚举、结构、联合类型的数据，以及由用户或系统定义的类（类型）的对象，都可以组成数组。
- 数组可以是多维的，按照其维度可以称为一/二/三/...维数组。

一维数组

- 由 n 个同一类型数据组成的一维序列，构成一维数组，一维数组的下标为 $0, \dots, n-1$ 。
- 任何一种已知类型数据都可说明为数组，数组说明的格式为：
 <类型名><数组名>[<元素数>]={<初值表>};
- 类型名：可以是基本类型名、派生类型名、类名、枚举类型名、结构、联合类型名（也可以是枚举、结构联合的类型说明）。
- 数组名：标识符，用于表示数组元素的下标变量。如 `array[2]` 表示数组的第 3 个元素。**同时也是一个指向数组的首元指针。**
- 元素数：一个正整数，指出数组的元素个数或数组的大小（size）。元素数要用方括号“`[]`”括起来。**方括号“`[]`”不可缺省，元素数则有时可缺省**，省略时必须赋初值，系统按所赋的初值个数确定数组的大小。
- 初值表：可缺省。是由**花括号“`{}`”**括起来且用逗号“`,`”分开的初始化常量值。

一维数组

- C++语言未提供对于数组整体进行操作的运算符和运算。对于数组的操作是通过对其元素，即下标变量进行的。下标变量的格式为：
 <数组名>[<下标>]
- 下标：整数或整型表达式，其取值范围为 $0 \sim n-1$ ， n =数组元素总数，例如：int A[4];共有 4 个下标变量：A[0]、A[1]、A[2]、A[3]。
- 对于下标变量可进行赋值和运算操作，操作类似于单个变量。
- 下标表达式应注意其值应保持在 **在 $0 \sim n-1$ 范围之内(从0开始)**。

多维数组

- 若干同一类型的数据形成 m 行 n 列的矩阵，则可称为二维数组，二维数组亦可视为一维数组的数组。比如 `int A[m][n]`; 说明了一个二维数组，它有 $m \times n$ 个元素，它也可以视为由 m 个一维数组 `int A[n]` 组成的（一维）数组。
- 类似地，还可以说明三维、四维数组，二维以上的数组统称为多维数组。多维数组的说明与一维数组类似（以二维为例）：

<类型名><数组名>[<行数>][<列数>]

- 例如： `char ch[2][3]`; 二维数组共有 2 行 3 列，元素个数为 $2 \times 3 = 6$ 。
- 在二维数组中，为数组赋初值的方式有两种：

`int a[2][3]={ {1, 2, 3}, {4, 5, 6} };`

`int a[2][3]={ 1, 2, 3, 4, 5, 6 }`

- 二者效果是一样的。系统将按逐行的次序，顺序为各元素赋值。由此可得，**多维数组在内存中的存储也是一维线性存储的。**

多维数组

- 对于多维数组的操作，同样是通过对其元素即下标变量的操作来进行的。操作中应注意**下标数量和表达式的取值范围**：
- 对于 `char A[m][n];` 的下标变量 `A[i][j]` 中 `i` 的值应在 $0 \sim m-1$ 之间，`j` 的值应在 $0 \sim n-1$ 之间。
- `A[i]` ($0 \leq i \leq m-1$) 表示的是一个一维数组，其元素个数为 `n`，这些元素可用下标变量：`A[i][0]`，`A[i][1]`，...，`A[i][n-1]`

数组与字符串

- 从表面上看，一个字符串就是一个字符数组，但在 C++ 语言中，二者并不完全相同，在涉及字符串处理时，必须注意，**字符串是一个以串尾符'\0'结尾的字符类型数组。**

```
char string1[7] = "China";
```

```
char string2[ ] = "China";
```

```
char string3[7] = { 'c', 'h', 'i', 'n', 'a' };
```

```
char string4[7] = { 'c', 'h', 'i' , 'n', 'a', '\0' };
```

- 这4个字符型数组是互不相同的。
- "China"是一个字符串常量，其字符串的长为5，由5个字符组成，但存放这一串常量却用6个字节，最后一个字节放串尾符'\0'，因此：字符数组 string1 的长度为7，字符数组 string2 的长度为6，字符数组 string3 的长度为7，赋初值后前5个元素的值为'c'、'h'、'i'、'n'和'a'。字符数组 string4 的长度为7，赋初值后应与 string1 相同。

目录

- 基本控制语句
 - 分支语句
 - 循环语句
 - 转向语句
- 导出数据类型
 - 数组
 - 结构类型

结构类型

- 数组是把若干相同类型的数据放在一起，结构则是把若干不同类型的数据放在一起。
- 例如，一个公司雇员的数据可能包括：名字，性别，工资，电话。可以定义为一个结构体表示。C++语言中的结构是作为类似于类的概念处理的，但是其概念上只是不同数据的联合体，不包含函数或属性，但是在C++支持给结构体定义函数。

```
cpurse2 > C structure-demo.h > ...
```

```
1  // structure-demo.cpp
2  #include <iostream>
3  using namespace std;
4  struct Employee {
5      char name[20];
6      enum sexType {male,female} sex;
7      float salary;
8      char phone[12];
9  };
```

结构类型

- 结构类型说明的格式为：

`struct <类型名> { <成员表> };`

- 成员表：<类型> <成员 1>;<类型> <成员 2>;...;<类型> <成员 n>; 可以看出成员表就是变量声明的列表。

cpurse2 > C structure-demo.h > ...

```
1  // structure-demo.cpp
2  #include <iostream>
3  using namespace std;
4  struct Employee {
5      char name[20];
6      enum sexType {male,female} sex;
7      float salary;
8      char phone[12];
9  };
```

结构类型

- 结构类型的变量说明的格式为：
[struct] <结构类型名> <变量名表>;
- **struct**关键字在申明变量时可以省略（和枚举类型一样），也可以在申明结构体的同时申明变量，变量名表与一般变量说明一样，赋初始值的方法同数据的赋初值方式。

cpurse2 > C structure-demo.h > ...

```
1  // structure-demo.cpp
2  #include <iostream>
3  using namespace std;
4  enum sexType {male,female};
5  struct Employee {
6      char name[20];
7      enum sexType {male,female} sex;
8      float salary;
9      char phone[12];
10 } gy0;
11 struct Employee gy1,gy2;
12 Employee gy3,gy4={"John Smith", male, 2107.5, "02223501234"};
```

结构类型

- 对结构类型的分量的存取由圆点运算符“.”实现，因此，为结构变量赋值，除了在变量说明中赋初值（如为 `gy4` 赋初值）的方法外，还可以用赋值语句或其他方式为结构分量赋值的方法：

```
11 void struct_test(){
12     struct Employee gy1,gy2;
13     Employee gy3,gy4={"John Smith", male, 2107.5, "02223501234"};
14     gy3.name="John";
15     gy3.sex=male;
16     cin>>gy3.salary;
17     cin>>gy3.phone;
18 }
```

- 无论采用哪种方法为结构分量赋值，都必须保证类型一致，比如上例中的字符串赋值在最新的C++编译器中会出错，提示不能用常量字符串赋值 `char` 数组。

```
structure-demo.h:24:13: error: incompatible types in assignment of 'const char [5]' to 'char [20]'
24 |     gy3.name="John";
   |     ~~~~~^~~~~~
```

结构类型

- 特别的，在以往版本中的C++编译器还允许直接对结构变量赋值，但是值得注意的是，这里的赋值在新版本中被更新为对赋值运算符=的重载。

```
23 void struct_test(){
24     struct Employee gy1,gy2;
25     Employee gy3,gy4={"John Smith", male, 2107.5, "02223501234"};
26     strcpy(gy3.name,"John");
27     gy3.sex=male;
28     cin>>gy3.salary>>gy3.phone;
29     gy1=gy3;
30     gy2=gy4;
31     cout<<"gy1"<<gy1<<endl;
32     cout<<"gy2"<<gy2<<endl;
33 }
```



```
constexpr Employee &Employee::operator=(const Employee &)
```

还有 1 个重载

```
$ g++ structure-demo.cpp -o structure-demo.exe
```

```
$ ./structure-demo.exe
```

```
31253.5 12315235
```

```
gy1{ name=John, sex=male, salary=31253.5, phone=12315235 }
```

```
gy2{ name=John Smith, sex=male, salary=2107.5, phone=02223501234 }
```

结构类型

- 此外，这个输出符号<<也是重载过的，具体重载如下。

```
23 void struct_test(){
24     struct Employee gy1,gy2;
25     Employee gy3,gy4={"John Smith", male, 2107.5, "02223501234"};
26     strcpy(gy3.name,"John");
27     gy3.sex=male;
28     cin>>gy3.salary>>gy3.phone;
29     gy1=gy3;
30     gy2=gy4;
31     cout<<"gy1"<<gy1<<endl;
32     cout<<"gy2"<<gy2<<endl;
33 }
```



```
13 ostream& operator<<(ostream& out, Employee eple){
14     const char* sex= eple.sex?"female":"male";
15     out<<"{ name="<<eple.name
16         <<", sex="<<sex
17         <<", salary="<<eple.salary
18         <<", phone="<<eple.phone
19         <<" }"<<endl;
20     return out;
21 }
```

```
$ g++ structure-demo.cpp -o structure-demo.exe
$ ./structure-demo.exe
31253.5 12315235
gy1{ name=John, sex=male, salary=31253.5, phone=12315235
gy2{ name=John Smith, sex=male, salary=2107.5, phone=02223501234
```

结构类型

- 结构类型的数据也可以组成数组，称为结构数组。结构数组在许多实际应用问题中经常用到，例如，通讯录、学生成绩单、商业销售记录、人事档案、资料登记表等。

```
5  enum sexType {male,female};
6  struct Employee {
7      char name[20];
8      enum sexType sex;
9      float salary;
10     char phone[12];
11 };
12 Employee efile[100];|
```

思考题

- switch 语句一般的使用格式是什么，其各 “<语句序列 S>”中是否必须包含 break 语句？
- for 语句的 3 个 “<表达式 E>”各自所起的作用是什么？缺省它们后的含义又各是什么？
- 任何一个 while 语句是否都可以用 for 语句来改写？任何一个 for 语句是否都可以用 while 语句来改写？
- do-while 语句与 while 语句有什么异同？
- “if (<表达式 E>) <语句 S>;”与 “while (<表达式 E>) <语句 S>;”的使用区别是什么？
- 数组的元素类型可以是一些什么数据类型？
- 任一个字符数组是否都是字符串？串尾符 '\0' 在一个字符数组中所起的作用是什么？
- 结构与数组的定义与使用有哪些异同？